Test Document

for

KaamSetu

• Domestic Work Marketplace

Version 1.0

Prepared by

Group #: 15 **Group Name:** ***NULL Pointers***

Ashish Gautam	**240210**	**ashishu24@iitk.ac.in**
Chaniyara Yug Nitesh	**240299**	**chaniyaray24@iitk.ac.in**
Harsh Vijay	**240436**	**harshvijay24@iitk.ac.in**
Harshit Saini	**240442**	**harshits24@iitk.ac.in**
Himanshu Yadav	**240455**	**himanshuy24@iitk.ac.in**
Ninad Tagade	**240699**	**ninadt24@iitk.ac.in**
Nirajkumar Binod Prasad	**240701**	**nirajkp24@iitk.ac.in**
Piyush Santosh Ghayal	**240748**	**santosh24@iitk.ac.in**
Vaidik	**241130**	**vaidik24@iitk.ac.in**
Virag Rathod	**241170**	**virag24@iitk.ac.in**

Course: **CS253**

Mentor TA: ***Utkarsh Agrawal***

Date: **03 / 04 / 2026**

CONTENTS II

REVISIONS II

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Version 1.0	Ashish Gautam Chaniyara Yug Harsh Vijay Harshit Saini Himanshu Yadav Ninad Tagade Nirajkumar Binod Prasad Piyush Santosh Ghayal Vaidik Virag Rathod	Information about the revision. This table does not need to be filled in whenever a document is touched, only when the version is being upgraded.	03/04/2026

1 Introduction

Test Strategy:

A hybrid testing strategy was adopted, combining both automated and manual testing approaches to ensure comprehensive validation of the application. Automated testing was performed using Jest to validate individual components such as middleware, models, and route logic through isolation and mocking techniques, while manual testing using Postman was used to verify API endpoints and overall system behaviour from a user perspective. This approach ensured both internal logic correctness and real-world functionality, providing a balanced and effective testing process.

Testing Timeline:

Testing was conducted primarily after the implementation phase was nearly complete, with a small portion of development continuing in parallel during testing.

Testers:

Testing was performed by the development team members themselves as part of an integrated development and testing workflow.

Coverage Criteria:

Test coverage was based on functional and structural criteria, including branch coverage, input validation, exception handling, and execution of critical logic paths.

Testing Tools Used:

Jest was used for automated unit testing, while Postman was used for API testing and system-level validation, providing an efficient and consistent testing environment across different stages of the application.

2 Unit Testing

Unit testing was focused on key components such as the authentication middleware, user model, and authentication routes, as these contain the core business logic, validation, and security mechanisms of the application. These modules involve complex operations like token verification, password hashing, and OTP handling, making them critical for unit-level validation. Other models and routes were not individually tested because they primarily implement standard CRUD operations and follow similar patterns, resulting in redundant test cases. Their functionality is sufficiently validated through the tested core components and is further covered during integration and system testing, ensuring comprehensive test coverage without unnecessary duplication.

1.1 MIDDLEWARE UNIT TESTING

1.1.1 Authentication Middleware – No Token Handling

This unit tests the behavior of the authentication middleware when no authorization token is provided in the request header. The middleware correctly blocks access and returns an unauthorized response.

Unit Details: authMiddleware function (/backend/middleware/auth.js)
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Tested the middleware behavior when no authorization header is provided. The system correctly returned a 401 status with message “No token provided”.

Structural Coverage: Branch coverage (missing token condition)

Additional Comments: Ensures that protected routes cannot be accessed without authentication credentials.

1.1.2 Authentication Middleware – Invalid Token Handling

This unit verifies how the middleware handles invalid or malformed JWT tokens. The middleware correctly identifies invalid tokens and prevents access.

Unit Details: authMiddleware function (/backend/middleware/auth.js)
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Simulated invalid JWT token. Middleware correctly handled the error and returned 401 with message “Invalid token”.

Structural Coverage: Exception handling (try-catch block)

Additional Comments: Validates system robustness against tampered or expired tokens.

1.1.3 Authentication Middleware – User Not Found Case

This unit tests the scenario where a valid token is provided but the corresponding user does not exist in the database. The middleware correctly denies access.

Unit Details: authMiddleware function (/backend/middleware/auth.js)
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When a valid token was provided but no user was found, the middleware returned a 401 status with “User not found”.

Structural Coverage: Conditional branch coverage (user existence check)

Additional Comments: Prevents unauthorized access using tokens of deleted or invalid users.

1.1.4 Authentication Middleware – Successful Authentication Flow

This unit tests the successful execution path where a valid token and user are provided. The middleware authenticates the user and allows request progression.

Unit Details: authMiddleware function (/backend/middleware/auth.js)
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Valid token and existing user resulted in successful authentication. The user object was attached to the request and next() was executed.

Structural Coverage: Happy path execution (full flow coverage)

Additional Comments: Confirms correct middleware behavior and proper request handling after authentication.

1.2 USER MODEL UNIT TESTING

1.2.1 User Model – Valid User Creation

This unit tests whether a user document with valid input fields can be successfully created and stored using the defined schema. The model correctly accepts valid data and saves it.

Unit Details: User model (models/User.js)
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: A user object with valid fields was created and saved successfully without errors.

Structural Coverage: Schema validation (valid input path)

Additional Comments: Confirms that the schema supports correct user creation.

1.2.2 User Model – Default Field Values Assignment

This unit verifies that default values defined in the schema are automatically assigned when not explicitly provided during user creation.

Unit Details: User model (models/User.js)
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Fields such as role and profileImage were automatically assigned default values (“user” and “profile.jpg”) when not provided.

Structural Coverage: Default value assignment coverage

Additional Comments: Ensures consistency and prevents missing field issues.

1.2.3 User Model – Rating Validation Constraints

This unit tests whether the schema correctly enforces constraints on rating values, ensuring they remain within the allowed range.

Unit Details: User model (models/User.js)
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Attempting to save ratings outside the allowed range (1–5) resulted in validation errors.

Structural Coverage: Validation rule coverage (min/max constraints)

Additional Comments: Prevents invalid rating data from being stored.

1.2.4 User Model – Skills Array Handling

This unit tests whether the schema correctly accepts and stores an array of skills as defined.

Unit Details: User model (models/User.js)
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Skills provided as an array of strings were successfully stored and retrieved without modification.

Structural Coverage: Array field handling

Additional Comments: Confirms proper handling of multi-value attributes.

1.3 AUTHENTICATION ROUTES UNIT TESTING

A. OTP Module

1.3.1 Send OTP – Invalid Email Input Handling

This unit tests how the system handles invalid email inputs during OTP generation. The system correctly validates the input and prevents OTP generation for invalid email formats.

Unit Details: send-otp route (/send-otp) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When an invalid email was provided, the system returned a 400 status with message “Invalid email”.

Structural Coverage: Input validation branch coverage

Additional Comments: Ensures only valid email formats are accepted for OTP generation.

1.3.2 Send OTP – Relay Failure Handling

This unit tests the behavior of the OTP system when the external relay server fails to send the OTP. The system correctly handles the failure and returns an error response.

Unit Details: send-otp route (/send-otp) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When the relay server request failed, the system returned a 500 status with message “Failed to send OTP”.

Structural Coverage: Exception handling (async external API failure)

Additional Comments: Validates system resilience against third-party service failures.

1.3.3 Send OTP – Successful OTP Generation and Storage

This unit verifies that OTP is generated, stored temporarily, and successfully sent via the relay server.

Unit Details: send-otp route (/send-otp) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Valid email input resulted in successful OTP generation, storage, and response message “OTP sent”.

Structural Coverage: Happy path execution (full flow)

Additional Comments: Confirms correct OTP lifecycle from generation to dispatch.

B. Registration Module

1.3.4 User Registration – Existing User Check

This unit tests whether the system correctly identifies and prevents registration when a user with the same email or phone number already exists.

Unit Details: register route (/register) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When an existing user was found in the database, the system returned a 400 status with message “User already exists”.

Structural Coverage: Conditional branch coverage (existing user check)

Additional Comments: Ensures uniqueness constraints are enforced at application level.

1.3.5 User Registration – Invalid OTP Verification

This unit verifies that registration is blocked when the provided OTP does not match the stored OTP.

Unit Details: register route (/register) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When an incorrect OTP was provided, the system returned a 400 status with message “Invalid OTP”.

Structural Coverage: Conditional branch coverage (OTP validation)

Additional Comments: Prevents unauthorized registrations without proper verification.

1.3.6 User Registration – Successful User Creation

This unit tests the complete successful flow of user registration including password hashing and database storage.

Unit Details: register route (/register) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Valid input resulted in successful user creation, password hashing, and response message “User registered successfully”.

Structural Coverage: Happy path execution (full flow)

Additional Comments: Confirms correct integration of validation, hashing, and persistence logic.

1.3.7 User Registration – Skills Normalization Logic

This unit tests whether the system correctly converts skills provided as a comma-separated string into an array format.

Unit Details: register route (/register) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Skills provided as a string were successfully converted into an array and stored correctly.

Structural Coverage: Data transformation logic coverage

Additional Comments: Ensures consistent data format regardless of input type.

C. Login Module

1.3.8 User Login – User Not Found Case

This unit tests the behavior of the login system when a user with the provided phone number does not exist. The system correctly prevents login and returns an error response.

Unit Details: login route (/login) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When a non-existent phone number was used, the system returned a 400 status with message “User not found”.

Structural Coverage: Conditional branch coverage (user existence check)

Additional Comments: Ensures invalid users cannot access the system.

1.3.9 User Login – Incorrect Password Handling

This unit verifies that the system correctly handles incorrect password attempts and prevents unauthorized access.

Unit Details: login route (/login) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When an incorrect password was provided, the system returned a 400 status with message “Wrong password”.

Structural Coverage: Conditional branch coverage (password comparison)

Additional Comments: Ensures password validation is properly enforced.

1.3.10 User Login – Successful Authentication and Token Generation

This unit tests the successful login flow, including password verification and JWT token generation.

Unit Details: login route (/login) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Valid credentials resulted in successful login, token generation, and user data returned without password field.

Structural Coverage: Happy path execution (full flow)

Additional Comments: Confirms correct authentication and secure response handling.

D. Reset Password Module

1.3.11 Reset Password – Invalid OTP Handling

This unit tests whether the system correctly rejects password reset requests when an invalid OTP is provided. The system ensures that only verified users can reset their passwords.

Unit Details: reset-password route (/reset-password) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When an incorrect OTP was provided, the system returned a 400 status with message “Invalid OTP”.

Structural Coverage: Conditional branch coverage (OTP validation)

Additional Comments: Ensures password reset security by enforcing OTP verification.

1.3.12 Reset Password – User Not Found Case

This unit tests the scenario where the provided email does not correspond to any existing user. The system correctly prevents password reset for non-existent users.

Unit Details: reset-password route (/reset-password) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When no user was found for the given email, the system returned a 400 status with message “User not found”.

Structural Coverage: Conditional branch coverage (user existence check)

Additional Comments: Prevents invalid password reset attempts.

1.3.13 Reset Password – Successful Password Update

This unit tests the successful password reset flow, including OTP verification, password hashing, and updating the user record.

Unit Details: reset-password route (/reset-password) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Valid OTP and existing user resulted in successful password update and response message “Password updated successfully”.

Structural Coverage: Happy path execution (full flow)

Additional Comments: Confirms correct integration of OTP validation, hashing, and database update.

E. Profile Update Module

1.3.14 Profile Update – Missing User ID Handling

This unit tests how the system behaves when the required user ID is not provided in the request. The system correctly prevents the update operation.

Unit Details: update-profile route (/update-profile) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When no user ID was provided, the system returned a 400 status with message “Missing user ID”.

Structural Coverage: Input validation branch coverage

Additional Comments: Ensures mandatory fields are validated before processing.

1.3.15 Profile Update – User Not Found Case

This unit tests the scenario where the provided user ID does not match any existing user. The system correctly prevents updates for invalid users.

Unit Details: update-profile route (/update-profile) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When no user was found for the given ID, the system returned a 404 status with message “User not found”.

Structural Coverage: Conditional branch coverage (user existence check)

Additional Comments: Prevents updates on non-existent user records.

1.3.16 Profile Update – Basic Information Update (Name/Address)

This unit tests whether basic user information such as name and address can be successfully updated.

Unit Details: update-profile route (/update-profile) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Valid user ID with updated name/address resulted in successful update and response message “Profile updated”.

Structural Coverage: Field update execution path

Additional Comments: Confirms proper handling of partial updates.

1.3.17 Profile Update – Skills String to Array Conversion

This unit tests whether skills provided as a comma-separated string are correctly converted into an array format before storage.

Unit Details: update-profile route (/update-profile) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: Skills provided as a string were successfully converted into an array and stored correctly.

Structural Coverage: Data transformation logic coverage

Additional Comments: Ensures consistent data format regardless of input type.

1.3.18 Profile Update – Profile Image Update Handling

This unit tests the behavior of the system when a new profile image is uploaded, including replacement of the old image and updating the file reference.

Unit Details: update-profile route (/update-profile) in auth.js
Test Owner: Team #15
Test Date: 29/03/2026 - 29/03/2026

Test Results: When a new image file was provided, the system successfully updated the profile image and handled previous file cleanup logic.

Structural Coverage: File handling and conditional branch coverage

Additional Comments: Validates correct integration of file upload and storage logic.

3 Integration Testing

1. Login Process

The integration between frontend login interface and backend authentication API was tested to ensure secure user login and session handling. The process worked correctly with proper validation and token generation.

Module Details: User Login (Frontend + /api/auth/login)

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- Login request correctly sent from frontend to backend
- JWT token generated and stored successfully
- Invalid credentials show proper error messages
- Successful login redirects user to main dashboard
- Session persists across navigation

Additional Comments: The integration is stable and secure. Minor delay may occur due to network latency during authentication.

2. Registration Process

The integration between registration form and backend APIs was tested to verify user creation and OTP-based verification. The workflow completed successfully with correct data storage.

Module Details: User Registration + OTP Verification

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- Registration form successfully sends data to backend
- OTP generation and verification works end-to-end
- User data stored correctly in database
- Duplicate email registration is prevented
- Error messages are properly displayed

Additional Comments: The OTP-based verification enhances security. Slight delays may occur due to external OTP service dependency.

3. Forgot Password Process

The password reset functionality was tested to ensure secure OTP-based recovery and correct password update. The integration worked as expected.

Module Details: Password Reset using OTP

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- OTP sent successfully to registered email
- OTP verification allows password reset
- Password updated correctly in database
- Invalid/expired OTP is rejected
- Secure reset flow maintained

Additional Comments: The reset process is secure and reliable. Performance depends on OTP delivery service.

4. Post Job

The integration between job posting UI and backend job creation API was tested to verify job submission and storage. The process successfully created jobs with proper validation.

Module Details: Job Creation (/api/jobs/create)

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- Job details successfully sent from frontend to backend
- Job stored correctly in MongoDB
- Validation for required fields works properly
- UI updates after successful job posting
- Invalid inputs handled with proper error messages

Additional Comments: The module performs reliably. Data validation at backend ensures consistency.

5. Live Jobs

The integration between backend job fetching API and frontend job listing screen was tested to ensure correct data retrieval and display. The system worked reliably.

Module Details: Job Fetching (/api/jobs/live)

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- Jobs fetched correctly from backend
- Newly posted jobs appear instantly
- Job details displayed correctly in UI
- No data inconsistency observed
- Error handling works for failed API calls

Additional Comments: The system performs efficiently with minimal delay in job updates.

6. Filters (Search & Sorting)

The filtering functionality was tested to ensure proper interaction between frontend filters and backend query handling. Results were accurate and consistent.

Module Details: Job Filtering (category, location, schedule, budget)

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- Filters correctly applied on job listings
- Category and location filtering works accurately
- Budget and schedule filters refine results properly
- Combined filters work without conflict
- Invalid filter inputs handled gracefully

Additional Comments: Filtering logic is efficient and scalable for larger datasets.

7. Job Acceptance

The integration between application selection and job status update APIs was tested to verify correct workflow after accepting a worker. The system behaved as expected.

Module Details: Accept Application (/api/applications/accept)

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- Application acceptance updates job status to “in_progress”
- Other applications are automatically rejected
- Changes reflected in both worker and customer dashboards
- Backend consistency maintained
- Unauthorized access prevented

Additional Comments: Atomic updates ensure data integrity across modules.

8. Referral System

The referral mechanism was tested to ensure proper OTP verification and onboarding of referred workers. The end-to-end flow worked successfully.

Module Details: Referral + OTP Verification (/api/referral)

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- Referral creation works successfully

- OTP sent and verified correctly
- Referred users can access and apply for jobs
- Invalid OTP is rejected
- Full referral workflow functions end-to-end

Additional Comments: The feature depends on external OTP services but remains stable overall.

9. Chat System

The integration between chat UI and backend messaging APIs was tested to verify real-time communication between users. The system performed reliably.

Module Details: Chat (/api/chat)

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- Chat enabled after job application/selection
- Messages sent and received correctly
- Chat history stored and retrieved properly
- Real-time communication works with minimal delay
- Unauthorized chat access restricted

Additional Comments: Minor latency observed under high load, but overall performance is smooth.

10. Profile Management

The integration between profile UI and backend APIs was tested to ensure correct fetching and updating of user information. The functionality worked smoothly.

Module Details: View & Update Profile

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- Profile data fetched correctly from backend
- Updates reflected instantly in UI and database
- Validation rules applied correctly
- Incorrect inputs handled properly
- Data consistency maintained

Additional Comments: The module is stable with consistent synchronization between frontend and backend.

11. Rating & Feedback

The rating system integration was tested to verify feedback submission and its reflection in user profiles. The feature worked correctly.

Module Details: Rating System after Job Completion

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- Ratings stored correctly in database
- Feedback displayed in user profiles
- Only completed jobs allow rating submission
- Ratings influence worker profile visibility
- Invalid submissions prevented

Additional Comments: This module enhances trust and transparency in the platform.

12. Account Section (My Jobs, Applications, Referrals)

The integration of dashboard components was tested to ensure correct aggregation and display of user-related data. All sections worked consistently.

Module Details: User Dashboard (Applications, Jobs, Referrals)

Test Owner: Team #15
Test Date: 30/03/2026 - 30/03/2026

Test Results:

- My Applications section shows correct applied jobs
- My Jobs section displays posted jobs accurately
- Referral data displayed correctly
- Status updates (pending/accepted/completed) are synchronized
- Data fetched correctly across all sections

Additional Comments: The dashboard provides a unified and consistent view of user activity.

4 System Testing

1. Requirement: F1 – Work Request Management

Tested whether a user can create, view, and cancel a work request successfully.

Test Owner: Team #15
Test Date: 31/03/2026 - 31/03/2026

Test Results: User successfully created a work request with valid inputs. The request appeared in the live jobs section. Cancellation worked correctly when no worker was finalized.

Additional Comments: Input validation and form handling were functioning correctly.

2. Requirement: F2 – Search and Filter Work Requests

Tested whether workers can search and filter available jobs based on category, location, schedule, and budget.

Test Owner: Team #15
Test Date: 01/04/2026 - 01/04/2026

Test Results: Workers were able to view all active jobs and apply filters successfully. Filter results matched expected criteria.

Additional Comments: Filtering performance was efficient and responsive.

3. Requirement: F2 – Apply to Work Request

Tested whether a worker can apply to a job and submit a proposal.

Test Owner: Team #15
Test Date: 01/04/2026 - 01/04/2026

Test Results: Worker successfully applied to a job. The application was visible in the applicant list. Withdrawal worked correctly before final selection.

Additional Comments: Proposal details were stored and retrieved correctly.

4. Requirement: F3 – Referral of Worker

Tested whether a user or worker can refer another worker and the referral behaves like a normal application.

Test Owner: Team #15
Test Date: 01/04/2026 - 01/04/2026

Test Results: Referral was successfully created and appeared in the applicant list. Referral behaved consistently with normal applications.

Additional Comments: Referral integration with application flow was seamless.

5. Requirement: F4 – View and Review Applications

Tested whether users can view all applications along with worker details.

Test Owner: Team #15
Test Date: 03/04/2026 - 03/04/2026

Test Results: Applications were displayed correctly with all required details such as worker name, rating, location, and expected pay.

Additional Comments: UI presentation was clear and user-friendly.

6. Requirement: F4 – Chat and Negotiation

Tested whether chat functionality allows communication between user and worker before job completion.

Test Owner: Team #15
Test Date: 31/03/2026 - 01/04/2026

Test Results: Chat functionality worked as expected. Messages were exchanged successfully between user and worker.

Additional Comments: Real-time communication was stable.

7. Requirement: F4 – Flexible Worker Selection (Modified Behavior)

Tested whether selecting a worker allows flexibility to change selection and keeps the job active until completion.

Test Owner: Team #15
Test Date: 02/04/2026 - 03/04/2026

Test Results: Upon selecting a worker, the selected worker was highlighted while other applicants remained visible. The user was able to change the selected worker before job completion. The job remained active in live jobs until marked as completed.

Additional Comments: This implementation improves flexibility and supports real-world negotiation scenarios.

8. Requirement: F5 – Rating and Review System

Tested whether users and workers can rate and review each other after job completion.

Test Owner: Team #15
Test Date: 03/04/2026 - 03/04/2026

Test Results: Ratings and reviews were successfully submitted and reflected on respective profiles.

Additional Comments: Rating aggregation was accurate.

9. Requirement: F6 – Work History Management

Tested whether completed jobs are stored and displayed in user and worker history.

Test Owner: Team #15
Test Date: 01/04/2026 - 03/04/2026

Test Results: Completed jobs were correctly recorded and displayed in history sections for both users and workers.

Additional Comments: Data persistence worked reliably.

10. Requirement: User Authentication – Login and Registration

Tested whether users can register and log in securely.

Test Owner: Team #15
Test Date: 31/03/2026 - 02/04/2026

Test Results: Users successfully registered and logged in with valid credentials. Invalid login attempts were rejected.

Additional Comments: Authentication flow was secure and consistent.

11. Requirement: Profile Management

Tested whether users can view and update profile details.

Test Owner: Team #15
Test Date: 01/04/2026 - 03/04/2026

Test Results: Profile updates were successfully saved and reflected immediately.

Additional Comments: Input validation prevented incorrect data entry.

12. Requirement: Non-Functional – Performance (Latency)

Tested whether the system responds within acceptable time limits.

Test Owner: Team #15
Test Date: 31/03/2026 - 01/04/2026

Test Results: Most operations completed within 1 second under normal conditions.

Additional Comments: Minor delays observed under peak load conditions.

13. Requirement: Non-Functional – Scalability

Tested whether the system can handle multiple users simultaneously.

Test Owner: Team #15
Test Date: 31/03/2026 - 03/04/2026

Test Results: System handled multiple concurrent users without crashes or failures.

Additional Comments: Performance degraded slightly under heavy load but remained functional.

14. Requirement: Non-Functional – Security

Tested whether user data is protected and unauthorized access is restricted.

Test Owner: Team #15
Test Date: 31/03/2026 - 03/04/2026

Test Results: Unauthorized access attempts were blocked. User data remained secure.

Additional Comments: Security mechanisms were functioning as expected.

5 Conclusion

How Effective and exhaustive was the testing?

The testing process was effective in validating the core functionality, logic, and security aspects of the application. Critical components such as authentication, user management, and middleware were thoroughly tested using both automated and manual approaches. By covering multiple test scenarios including edge cases, error handling, and successful flows, the testing ensured a high level of reliability and correctness. While not every module was tested individually at the unit level, the overall testing approach was sufficiently exhaustive due to coverage of core logic and end-to-end validation.

Which components have not been tested adequately?

Some modules, particularly those related to auxiliary features such as job management, chat functionality, and referral systems, were not tested extensively at the unit level. These components mainly involve standard CRUD operations and follow similar implementation patterns, making their individual unit testing less critical. However, they were indirectly validated through integration and system testing, ensuring their functional correctness within the overall application workflow.

What difficulties have you faced during testing?

One of the primary challenges faced during testing was handling dependencies such as database operations, authentication mechanisms, and external services like OTP relay systems. Mocking these components for unit testing required additional effort and understanding. Additionally, coordinating testing alongside ongoing development posed minor challenges, as some features were still being refined during the testing phase. Ensuring consistency across different modules developed by multiple team members was also a notable difficulty.

How could the testing process be improved?

The testing process could be improved by incorporating more comprehensive automated testing across all modules, including less critical components, to achieve higher coverage. Introducing dedicated integration and end-to-end testing tools would further enhance validation of complete workflows. Additionally, adopting a more test-driven or continuous testing approach during development, rather than primarily after implementation, would help identify issues earlier and improve overall code quality.

Appendix A - Group Log

Date	Progress
29/03/2026	Unit testing was conducted for core components including middleware, user model, and authentication routes. Test cases were implemented and executed using automated testing.
30/03/2026	Integration testing was performed to validate interactions between different modules and API endpoints. Data flow and combined functionality across components were verified.
31/03/2026	System testing was initiated, focusing on overall application behavior and API validation using Postman. Initial workflows were tested.
01/04/2026	Continued system testing with validation of additional user flows and identification of minor issues in application behavior.
02/04/2026	Further system testing and refinement, including edge case handling and verification of fixes.
03/04/2026	Final system testing, documentation completion, and consolidation of test results for submission.